Keywords: test-time reasoning • self-consistency • error localization • token efficiency

StepBack: Step-Level Error-Localized Resampling for Efficient Test-Time Reasoning

Localizing the First Error Step and Resampling Only Its Continuation Cuts Token Usage by Up to 41% at No Accuracy Cost

By Yang Ouyang, Jung-Eun Kim

arXiv:2611.08403 • cs.AI / cs.LG • NeurIPS 2026

Self-consistency generates many full chain-of-thought trajectories and takes a majority vote, but every chain runs to completion regardless of where the error occurs. StepBack localizes the most likely first error using lightweight signals and resamples only from that step onward, matching self-consistency accuracy on seven benchmarks while reducing token use by up to 41.26%.

AT A GLANCE

Problem: Self-consistency is expensive because every reasoning trajectory runs to completion, even when an error occurs early.

Why it matters: Test-time compute budgets are a primary cost driver for deployed LLM reasoning; a method that preserves accuracy at lower token cost directly reduces inference cost.

Method: Localize the first likely error in draft trajectories using NLL and PRM signals, then resample only the erroneous step and its continuation, finishing with majority voting.

Result: Matches SC accuracy on 7 benchmarks; cuts token usage by up to 41.26%.

Evidence: GSM8K, MATH, AIME, MultiArith, HotpotQA, 2WikiMultiHopQA, MuSiQue; Llama-3.1-8B-Instruct backbone.

The Big Picture

Chain-of-thought reasoning substantially improves LLM accuracy by externalizing intermediate steps. Self-consistency (SC) amplifies this further: by sampling many independent trajectories and taking a majority vote, SC cancels out reasoning errors that occur in different places on different runs. The cost is linear in the number of trajectories—a budget of 32 trajectories means 32× the tokens of a single greedy answer.

The key observation of StepBack is that this cost model is wasteful. Errors in a chain-of-thought trajectory are not uniformly distributed across steps; they tend to cluster in the early-to-middle portion of a chain. Once an error is made, subsequent steps correctly propagate it to a wrong final answer. Regenerating the entire trajectory from scratch discards all the correct prefix steps.

Why Existing Approaches Fall Short

Self-consistency treats every trajectory uniformly and cannot distinguish high-quality from low-quality prefixes. **Best-of-N** and verifier-guided methods re-rank complete trajectories after generation but still generate all N full chains. **Adaptive Consistency** reduces the trajectory count by stopping early when the answer distribution is sufficiently peaked. StepBack and Adaptive Consistency are orthogonal: StepBack shortens each trajectory, while Adaptive Consistency reduces the trajectory count. The two compose.

Core Mechanism

Given *m* draft trajectories, StepBack splits each into atomic steps using the model's natural delimiter tokens. For each step two signals are computed: (1) NLL signal—the mean negative log-likelihood of tokens in the step under the model; and (2) PRM signal—the process reward model score estimating the probability that all steps up to and including this one are correct.

The *anchor* step is the earliest step flagged by NLL > τ_{NLL} = 0.1 or PRM < τ_{PRM} = 0.05. For each flagged trajectory, $k = 2$ fresh continuations are sampled from the anchor step onward, keeping the prefix fixed. Final answers from all trajectories are pooled and the plurality answer is returned.

Technical Formulation

Let m = 16 draft trajectories be given. For trajectory τ_i, the prefix up to step $a_i - 1$ is p_i and resampled continuations are drawn from:

$$\tilde{c}_{ij} \sim p_\theta(\cdot \mid x, p_i), \quad j = 1, \ldots, k$$

The expected token count is $\mathbb{E}[\text{tokens}] = m \cdot L_{\text{prefix}} + \alpha \cdot k \cdot L_{\text{suffix}}$, where α is the fraction of trajectories flagged and L_{prefix}, L_{suffix} are mean prefix and suffix lengths. When errors cluster early, L_{prefix} is small and savings are largest.

Learning or Inference Procedure

StepBack is a pure inference-time method; no model parameters are modified. Required inputs: a base LLM for draft generation, a process reward model (PRM) trained to score individual reasoning steps (a Llama-3.1-8B model fine-tuned on PRM800K), and two scalar thresholds τ_{NLL} = 0.1 and τ_{PRM} = 0.05 tuned on a small validation split.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is: on the seven benchmarks tested, StepBack's accuracy is not statistically significantly different from SC at the same sampling budget of m = 32 effective trajectories. The token-reduction claim (up to 41.26%) is an empirical upper bound observed on the MATH benchmark.

Experimental Findings

- **Accuracy:** Statistically indistinguishable from SC on all 7 benchmarks.
- **Token reduction:** 20–41% fewer tokens; largest on MATH (41.26%) and 2WikiMultiHopQA (38.1%).
- **Composition:** StepBack + Adaptive Consistency achieves >41% token reduction at equal or slightly higher accuracy.
- **Threshold sensitivity:** Accuracy stays within 1% across the full threshold grid.

Ablations and Interpretation

NLL-only vs. PRM-only vs. combined: NLL alone reduces tokens by 28%; PRM alone by 31%; combined by 37% on average. The two signals flag different steps on roughly 20% of trajectories, confirming complementarity.

Resampling k: k = 1 achieves 90% of token savings at a 1–2% accuracy drop; k = 3 shows no gain over k = 2.

PRM quality: Replacing the trained PRM with zero-shot self-evaluation reduces accuracy by 2.1% and token savings by 8%.

Reference

Yang Ouyang, Jung-Eun Kim. **StepBack: Step-Level Error-Localized Resampling for Efficient Test-Time Reasoning.** arXiv:2611.08403. NeurIPS 2026. arxiv.org/abs/2611.08403

Keywords: evolutionary optimization • latent agents • program synthesis • scientific computing

SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization

Agents That Reason in Activation Space Rather Than Text Evaluate 5–13× More Programs Per Hour on Mathematical and Scientific Discovery

By Qile Jiang, George Em Karniadakis
arXiv:2611.08483 • cs.AI / cs.NE • Brown University

LLM-guided evolutionary search is accurate but slow because each candidate requires a full chain-of-thought forward pass. SwarmEvolve replaces text-decoding agents with latent agents that share KV caches and reason in activation space, evaluating 5–13× more programs per hour than state-of-the-art frameworks and reaching equivalent solution quality in 3–8 minutes versus 60 minutes for competing systems.

AT A GLANCE

Problem: LLM-guided evolutionary search is accurate but too slow for large-scale discovery; each round requires full chain-of-thought for every candidate.

Why it matters: Scientific computing and mathematical discovery require evaluating hundreds of program variants per experiment; throughput is a hard bottleneck.

Method: 10 swarms of 45 latent agents share KV caches and reason for 40 latent steps without decoding to text; a lightweight decoding head maps top agent states to program tokens.

Result: 5–13× more programs per hour; solution quality reached in 3–8 min vs. 60 min for text-based frameworks.

Evidence: 21 problems spanning symbolic math and PDE solver kernel optimization; baselines: OpenEvolve, ShinkaEvolve, CodeEvolve.

The Big Picture

Program evolution with LLMs (OpenEvolve, FunSearch, ShinkaEvolve) has produced surprising discoveries in combinatorics and numerical computing by using LLMs as intelligent mutation operators. The approach works because LLMs encode prior knowledge about program structure, algorithm design, and mathematical transformations that random mutation cannot access.

The fundamental bottleneck is throughput. Generating a new program candidate with chain-of-thought reasoning costs one full LLM forward pass with a long context window. At 450 tokens per second on typical hardware, a 1,000-token chain-of-thought allows only 27 candidates per minute per GPU. SwarmEvolve attacks this bottleneck by eliminating the decoding step from most of the computation.

Why Existing Approaches Fall Short

Text-based frameworks (OpenEvolve, CodeEvolve) generate each candidate as a full text completion—flexible but slow. **Parallel sampling with temperature** can explore multiple candidates simultaneously but each sample is independent and cannot condition on what other concurrent samples produce. **Smaller surrogate models** are faster but lack reasoning quality, leading to lower-quality proposals and slower convergence.

Core Mechanism: Latent Agents with KV Sharing

Each agent is a frozen LLM conditioned on a shared prefix KV cache covering the problem description and global best solution. The agent then runs 40 autoregressive steps *without* applying the unembedding matrix: no token is produced. Instead, the final hidden state $h_i \in \mathbb{R}^d$ encodes the agent's proposal in activation space.

Agents in the same swarm share a common prefix KV cache, eliminating redundant computation. Different random seed vectors (appended to the shared prefix) induce diversity within the swarm. A lightweight decoding head maps the top-*r* agents' hidden states to program token sequences.

Technical Formulation

Let K = 10 swarms each containing N = 45 agents. The shared prefix for swarm k is:

```
K_k = KVCache([problem, x*_k])
```

where x^*_k is the globally best program found so far. Each agent produces:

```
h_{k,i} = f_θ(z_{k,i}, K_k) ∈ ℝ^d
```

The fitness-weighted swarm centroid is `ħ_k = ∑w_i h_{k,i} / ∑w_i` with weights `w_i = softmax(F_{k,i}/T)`. The top-r = 5 agents by cosine similarity to the centroid are decoded.

Learning or Inference Procedure

The decoding head D_φ is a 2-layer MLP trained offline on (hidden-state, program) pairs extracted from text-based evolutionary runs. At inference time D_φ is frozen; SwarmEvolve requires no online training. The evolutionary loop per round: (1) Update all K prefix caches; (2) Run all 450 agents in parallel (40 latent steps); (3) Select top-r agents per swarm; (4) Decode to programs via D_φ; (5) Execute and score; (6) Update global bests.

What the Guarantee Says

No formal convergence guarantee is provided. The empirical claim is throughput superiority on 21 specific problems. Best-known solutions to 11 of the 21 problems (symbolic math subset) have formal certificates; SwarmEvolve recovers certified-correct solutions for 9 of these 11 within the 2 GPU-hour budget.

Experimental Findings

- **Throughput:** 5–13× more programs per hour than all three baselines.
- **Time-to-solution:** Median 3–8 minutes vs. 60 minutes on the same hardware.
- **Quality:** Highest average score across 21 problems; wins 17 of 21 head-to-head.
- **Diversity:** 10-swarm config produces 1.9× more distinct program families per hour than a 450-agent single-swarm variant.

Ablations and Interpretation

Latent steps: 40 is near-optimal; 20 steps reduces quality by 4.3 points; 80 shows no improvement. **Swarm count:** 10 swarms outperform 5 by 2.7 points and 1 swarm by 7.1 points. **KV sharing:** Removing the shared prefix cache reduces throughput 2.3× and quality 3.2 points. **Decoding head:** Replacing D_φ with a linear probe reduces program validity from 94% to 71%.

Reference

Qile Jiang, George Em Karniadakis. **SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization.** arXiv:2611.08483. arxiv.org/abs/2611.08483

Keywords: LLM inference • quantization • memory hierarchy • KV cache

StrataFlux: Memory-Hierarchy-Native Low-Bit LLM Inference

Using Host DRAM and NVMe SSD as Precision Strata Recovers Near-Full Accuracy at 2-Bit Weight Quantization Without Extra GPU Memory

By Jake Hyun, Ziyu Zhao, Jae-sun Seo, Mohamed S. Abdelfattah

arXiv:2611.08363 • cs.LG / cs.DC • submitted 5 Nov 2026

StrataFlux recovers accuracy lost to 2–4-bit quantization by treating the full memory hierarchy—GPU HBM, host DRAM, and NVMe SSD—as a stratified precision resource. On 2-bit Llama-3.1-8B-Instruct, WikiText-2 perplexity drops from 9.46 to 8.40 at 95% base throughput; on 2-bit Qwen3-8B, AIME 2024/2025 accuracy rises from 11.0% to 47.1% with no additional GPU memory.

AT A GLANCE

Problem: 2-bit quantization reduces GPU memory but collapses accuracy; existing mixed-precision fixes keep all higher-precision weights on-GPU, negating the memory benefit.

Why it matters: Enabling 2-bit inference at acceptable accuracy would halve peak GPU memory for many deployments, making 70B+ models accessible on single-GPU servers.

Method: Stratified sensitivity profiling assigns weight layers to GPU HBM (2-bit), DRAM (4-bit), or NVMe SSD (8-bit); a prefetch scheduler hides tier-B/C transfer latency behind GPU compute.

Result: 2-bit Llama-3.1-8B perplexity 8.40 (vs. 9.46 baseline); 2-bit Qwen3-8B AIME accuracy 47.1% (vs. 11.0%).

Evidence: WikiText-2, AIME 2024/2025, 128K long-context; compared against 2-bit RTN, GPTQ, AWQ; RTX 5090.

The Big Picture

Large language model inference is bounded by memory bandwidth, not compute. Quantization reduces both peak memory and bandwidth requirements: 2-bit weights need 16× less memory than FP32. The problem is accuracy. At 2 bits, quantization error is large enough to damage reasoning ability, particularly on tasks requiring precise recall or long multi-step chains.

Prior solutions (GPTQ, AWQ, SpQR) identify sensitive weights and store them at higher precision *on the GPU*—but this cancels the memory benefit for the sensitive fraction. StrataFlux reframes the question: host DRAM is 8–32× larger and 5–10× slower; NVMe SSD is larger still. Sensitive weights that need 4- or 8-bit precision can live in DRAM or SSD and be streamed to GPU only when needed, overlapped with GPU computation on the previous layer.

Why Existing Approaches Fall Short

Uniform quantization (RTN, 2-bit) is fast and memory-efficient but loses too much accuracy in sensitive layers, especially attention projections and early embedding layers.

Mixed-precision on-GPU (GPTQ, AWQ, SpQR) keeps sensitive weights at 4–8 bits on the GPU. This recovers accuracy but requires extra GPU memory, partially defeating the purpose of quantization.

CPU offloading (FlexGen, DejaVu) moves entire layers to CPU, introducing large latency spikes. StrataFlux differs: it offloads only the *higher-precision copies* of sensitive layers, not the full layer; the base 2-bit copy always stays on GPU for fast path.

Core Mechanism: Three-Tier Precision

Stratum A (GPU HBM): Base quantized weights of all layers at the target bit-width. Always resident.

Stratum B (host DRAM): 4-bit copies of the top-k% most sensitive layers, paged in one block ahead of the layer it corrects.

Stratum C (NVMe SSD): 8-bit copies of an extremely sensitive subset (typically first and last 2 transformer blocks), paged asynchronously via direct DMA.

For a stratum-B/C layer, the forward pass combines the base 2-bit GPU weight with a residual correction loaded from DRAM or SSD: `y = W_base x + ΔW_tier x` where $\Delta W_{tier} = W_{tier} - W_{base}$. This residual form ensures an approximate result if the DMA transfer has not completed, instead of blocking.

Technical Formulation

Sensitivity of layer l is estimated by the Fisher trace:

```
S_l ≈ (1/|C|) ∑_{x∈C} ||W_l x − Ŵ_l x||^2
```

where C is a calibration corpus, W_l is full-precision weight, and $\hat{W}_l$ is 2-bit quantized. Layers are ranked by S_l and assigned by thresholds θ_B and θ_C, set by hardware memory budget constraints.

Learning or Inference Procedure

StrataFlux requires no fine-tuning. The workflow: (1) Run calibration pass on 128 sequences to compute S_l; (2) Assign layers to strata; (3) Quantize and place stratum-B copies in DRAM, stratum-C in SSD; (4) At inference, the prefetch scheduler issues DMA requests for the next 3 stratum-B layers and 1 stratum-C layer ahead of execution, overlapping transfers with GPU computation.

What the Guarantee Says

No theoretical accuracy bound is provided. The empirical claim is Pareto-dominance over existing mixed-precision methods: StrataFlux achieves lower perplexity at lower GPU memory than any configuration of GPTQ or AWQ tested. No latency bound under SSD-latency spikes is given; the base path ensures correctness but not accuracy.

Experimental Findings

- **Perplexity:** 2-bit Llama-3.1-8B WikiText-2 PPL 8.40 (StrataFlux) vs. 9.46 (2-bit RTN base); 95% base throughput on RTX 5090.
- **Reasoning:** 2-bit Qwen3-8B AIME 2024/2025 accuracy 47.1% vs. 11.0% (2-bit RTN); GPU memory unchanged.
- **Long-context:** 3-bit weights + 2-bit KV at 128K context matches vLLM 4-bit AWQ + 8-bit KV at half the GPU memory.
- **Throughput:** Prefetch scheduler achieves 95% of base 2-bit throughput by hiding DRAM latency.

Ablations and Interpretation

Stratum-C necessity: Removing SSD tier and assigning stratum-C layers to DRAM at 4-bit increases DRAM footprint by 18% at no accuracy change.

Residual form: Replacing residual correction with full replacement reduces throughput by 12% because the base 2-bit GPU path is always bypassed.

Calibration size: 32 sequences achieve 94% of sensitivity ordering quality of 512 sequences; 128 reaches 99%.

Model size: Relative perplexity improvement is larger for 8B models (1.1 PPL points) than 70B models (0.4 PPL points).

Reference

Jake Hyun, Ziyu Zhao, Jae-sun Seo, Mohamed S. Abdelfattah. **StrataFlux: Memory-Hierarchy-Native Low-Bit LLM Inference.** arXiv:2611.08363. arxiv.org/abs/2611.08363

Keywords: reinforcement learning • reward machines • vision-language models • non-Markovian tasks

RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines

An LLM Builds the Finite Automaton; a VLM Grounds Its Propositions; the Agent Earns Credit for History-Dependent Task Progress

By Roko Parac, Christos Ziakas, Leo Ardon

arXiv:2611.08308 • cs.AI • submitted 6 Nov 2026

Most VLM-based reward methods assume that reward depends only on the current observation. RM-VLM handles tasks that require history-dependent credit by having an LLM construct a reward machine from a text description and a VLM ground each proposition in image observations. Evaluated on grid navigation, robotic manipulation, and 3D object collection, RM-VLM consistently outperforms existing VLM-reward methods on task success.

AT A GLANCE

Problem: VLM reward methods assume Markovian tasks; real tasks often require ordered sub-goals whose credit depends on history.

Why it matters: Without history-aware reward, RL agents fail to distinguish correct-order completions from incorrect-order ones and cannot reliably learn ordered multi-step tasks.

Method: LLM decomposes task description into propositions and builds a reward machine (finite automaton); VLM grounds propositions in image observations; VLM value estimator provides dense shaping.

Result: Consistently higher task success than all baselines across three task domains.

Evidence: Grid-world navigation, robotic manipulation sequence, 3D object collection; baselines: VLM-RM, CLIP-reward, TAMER+VLM.

The Big Picture

VLM-based reward models automate reward specification by scoring image observations against a natural-language goal description. This works well for Markovian goals where reward depends only on the current state. It breaks for *ordered* tasks: “pick up the red block, then place it in the green zone, then open the door.” A VLM reward evaluating only the current image cannot distinguish whether the red block was placed *before* or *after* the green zone interaction.

Reward machines (Icarte et al., 2018) are the standard formalism for non-Markovian task structure: a finite automaton tracks which sub-goals have been achieved and transitions on logical propositions evaluated in the environment. RM-VLM combines the LLM’s ability to decompose natural-language task descriptions with the VLM’s ability to ground propositions in images.

Why Existing Approaches Fall Short

VLM scalar reward (VLM-RM) scores the current image against the full task description. It cannot distinguish sub-goal ordering and provides identical reward to agents that complete sub-goals in any order.

CLIP-reward embeds image and text goal in CLIP space. It has the same Markovian limitation and additionally fails to track incremental progress toward structured sub-goals.

Handcrafted reward machines require a domain expert to specify the propositions and automaton structure manually—the bottleneck RM-VLM eliminates.

Core Mechanism

Given task description T, an LLM produces: a set of Boolean propositions $\{p_1, \ldots, p_n\}$ over environment entities; and a finite automaton $M = (Q, q_0, q_f, \delta, \rho)$ where $\delta: Q \times 2^P \to Q$ is the transition function and $\rho: Q \times Q \to \Re$ assigns reward to transitions. The LLM outputs M as a JSON object, which is validated and compiled.

At each step, the VLM receives image o_t and each proposition p_i, returning $P(p_i=\text{true} \mid o_t)$. The resulting Boolean vector advances M’s state through δ, triggering the reward $\rho(q_t, q_{t+1})$ on any transition.

Technical Formulation

The reward machine defines a non-Markovian MDP with total reward:

```
R_M(s_t, a_t) = ρ(q_t, q_{t+1}) + r^+_t
```

where $q_{t+1} = \delta(q_t, L(o_t))$ and L is the VLM labelling function with threshold $\tau = 0.5$. The shaped bonus from a VLM value estimator is:

```
r^+_t = λ ∑_{p_i ∈ pending(q_t)} P(p_i=true | o_t)
```

The augmented state (s_t, q_t) is Markovian in this MDP, so standard policy-gradient methods (PPO, SAC) apply.

Learning or Inference Procedure

(1) Submit task description to LLM to obtain M as JSON. (2) Validate M (check q_f reachable, no duplicate states). (3) Initialize policy π_θ with PPO; use (s_t, q_t) as augmented state. (4) At each step, query VLM for $L(o_t)$, advance M, and compute R_M. (5) Train for 5M environment steps. No fine-tuning of the LLM or VLM is performed; both are frozen.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is consistent task-success superiority over all baselines in all three domains tested. No sample complexity bound is given.

Experimental Findings

- **Grid-world navigation:** RM-VLM 87% vs. VLM-RM 61%, CLIP-reward 54%, TAMER+VLM 70%.
- **Robotic manipulation sequence:** RM-VLM 74% vs. VLM-RM 38%, CLIP 29%, TAMER+VLM 51%; largest gap because out-of-order completions are highest in this domain.
- **3D object collection:** RM-VLM 81% vs. VLM-RM 58%.
- **Dense shaping:** Removing the shaped bonus reduces manipulation success from 74% to 52%.

Ablations and Interpretation

LLM construction errors: On 15% of task descriptions, the LLM produces a machine with at least one missing transition; retries resolve 90% of these cases.

VLM grounding accuracy: Replacing the VLM with oracle labels improves manipulation success from 74% to 88%, establishing an upper bound from grounding limitations.

Threshold τ: $\tau = 0.5$ optimal; $\tau = 0.3$ causes false-positive transitions; $\tau = 0.7$ misses valid ones. **Shaping coefficient λ:** $\lambda = 0.1$ is optimal; larger values cause reward-shaping pathology.

Reference

Roko Parac, Christos Ziakas, Leo Ardon. **RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines.** arXiv:2611.08308.
arxiv.org/abs/2611.08308

Keywords: medical AI • graph memory • clinical dialogue • patient state tracking

MedGraph: Maintaining Patient State with Graph Memory for Long-Term Medical Dialogue

Three-Layer Hierarchical Graph Memory Eliminates Outdated State, Cross-Patient Contamination, and Missing Safety Facts, Gaining 10.5 Points on MedMemoryBench

By Donghua Zhang, Zizheng Wang, Ze Dong, Yiding Feng, Siyu Qian
arXiv:2611.08433 • cs.CL / cs.AI • submitted 6 Nov 2026

Medical dialogue assistants fail in longitudinal settings because flat memory systems accumulate outdated states, mix facts across patients, and silently drop safety information. MedGraph introduces a three-layer hierarchical graph memory that makes clinical state transitions explicit and unconditionally prepends safety facts to every query. On MedMemoryBench it exceeds the strongest of five baselines by 10.4–10.5 percentage points with both GPT and Qwen backbones.

AT A GLANCE

Problem: Flat memory systems for clinical dialogue produce outdated state, cross-patient contamination, and silently missed safety facts across multi-visit conversations.

Why it matters: LLMs used as clinical assistants in longitudinal care must maintain accurate, up-to-date patient state; errors can cause harmful medication or diagnosis decisions.

Method: Three-layer memory: raw transcripts (L1), a typed clinical fact graph with temporal validity tags (L2), and a patient profile unconditionally attached to every query (L3).

Result: 10.4–10.5 pp above the strongest of five baselines on MedMemoryBench with GPT and Qwen backbones.

Evidence: MedMemoryBench (multi-visit clinical dialogues); baselines: flat vector, sliding window, RAG, Zep, MemoryOS.

The Big Picture

LLMs deployed as clinical dialogue assistants need to maintain accurate knowledge of a patient's medical state across many visits over weeks or months. Through error analysis of five baseline systems on longitudinal medical dialogues, the authors identify three recurring failure modes:

1. **Outdated state:** A resolved symptom is reported as currently active because an older transcript fragment scores higher in retrieval than the later discharge note.
2. **Cross-patient contamination:** Facts from a different patient's earlier session pollute the current patient's context when embedding similarity overrides patient identity.
3. **Missing safety facts:** A known allergy or contraindication is not retrieved because it was not in the top-k most similar fragments for the current query.

Why Existing Approaches Fall Short

Flat vector memory embeds all clinical utterances and retrieves top-k by similarity. It does not distinguish whether a fact is active or resolved, and similarity can produce facts from other patients.

Sliding-window context loses long-term facts (a years-old allergy, a chronic condition diagnosed at first visit) when they fall out of the window.

Zep (entity memory) maintains entity states but uses flat entity records without typed temporal transitions; it conflates active and resolved states. **MemoryOS** provides structured memory with hierarchical layers but does not specialize for clinical entity types or enforce mandatory safety-fact prepending.

Core Mechanism: Three-Layer Memory

Layer 1 — Raw dialogues. Full session transcripts stored verbatim, searched by BM25 retrieval for specific historical passages.

Layer 2 — Clinical fact graph. A typed knowledge graph over entities (symptoms, diagnoses, medications, lab results, procedures) with typed relations and temporal validity tags: *active*, *resolved*, or *historical*. Type-specific update rules: if an entity is new, add as active; if matched with a resolution signal, change tag to resolved; if repeated observation with no state change, update timestamp only.

Layer 3 — Patient profile (safety block). Per-patient compact record of allergies, contraindications, chronic conditions, emergency contacts. *Unconditionally prepended* to every query context, bypassing retrieval. Each patient has a fully separate graph instance, making cross-patient contamination architecturally impossible.

Technical Formulation

At query time the context assembled for the LLM backbone is:

```
C = Profile_p ⊕ Graph_p^{active} ⊕ RAG(D, q)
```

where Profile_p is the L3 safety block (mandatory), $\text{Graph_p}^{\text{active}}$ is the subgraph of currently active entities serialized as structured text, and RAG(D, q) is BM25-retrieved transcript fragments. The "resolved" signal is detected by a small classifier trained on clinical resolution phrases.

Learning or Inference Procedure

MedGraph uses no end-to-end learning. Components: a clinical NER/RE pipeline pre-trained on MIMIC-IV notes (entity and relation extraction); a resolution-signal classifier (BERT-based, fine-tuned on clinical resolution annotation); and an arbitrary LLM backbone (tested with GPT-4o-mini and Qwen2.5-72B). All components are frozen at inference time.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is superior clinical accuracy across all five baselines on MedMemoryBench with both LLM backbones. The error analysis identifies three failure modes and verifies per-category that MedGraph's design eliminates each by construction.

Experimental Findings

- **Overall accuracy (GPT-4o-mini):** MedGraph +10.5 pp above best baseline (Zep).
- **Overall accuracy (Qwen2.5-72B):** MedGraph +10.4 pp above best baseline.
- **Outdated-state errors:** Reduced by 68% relative to Zep.
- **Safety-fact recall:** 100% (by mandatory prepending) vs. 73–82% for retrieval-based baselines.
- **Cross-patient contamination rate:** 0% (by graph isolation) vs. 3–7% for vector-memory baselines.

Ablations and Interpretation

Without mandatory safety block: Safety-fact recall drops from 100% to 79%, confirming retrieval alone is insufficient.

Without typed temporal updates: Using simple entity overwrite increases outdated-state errors 3.2×.

Without graph isolation: Merging all patients into a single graph increases cross-patient contamination to 4.1%.

L1 retrieval necessity: Removing raw transcript retrieval reduces overall accuracy by 4.3 pp, showing verbatim historical passages provide useful evidence beyond the structured graph.

Reference

Donghua Zhang, Zizheng Wang, Ze Dong, Yiding Feng, Siyu Qian. **MedGraph: Maintaining Patient State with Graph Memory for Long-Term Medical Dialogue.** arXiv:2611.08433. arxiv.org/abs/2611.08433